Report: Comparative Analysis of RNN Architectures for Sentiment Classification

GitHub repository: https://github.com/anumsagheer01/imdb-seq-models

1 Introduction

This project is about training simple sequence models and seeing what really changes the results. I kept almost everything fixed and only changed one thing at a time. I looked at three sequence lengths and three architectures, few activations and optimizers, and whether gradient clipping helps. I report accuracy, macro F1, and average time per epoch so I can understand what works best on my CPU.

2 Dataset Summary

I used the IMDB movie reviews dataset with the standard split, which has twenty-five thousand reviews for training and twenty-five thousand for testing. My first step, before beginning to build my model, was to clean the text in a very basic way. I lowercased everything, removed punctuation and extra symbols, and turned line breaks into spaces. I kept only the ten thousand most frequent words so the vocabulary stays small on a CPU, since my computer couldn't compute a lot of data. Each review was then turned into a list of token IDs and padded or cut to a fixed length. I tried three lengths that are, 25, 50, and 100 tokens. From a quick look into what a review looks like, I know that the average review is much longer than 100 words, so truncation definitely happens but the point here is to see how much length I really need without making training slow. All runs used the same label format: positive vs negative.

3 Model Configuration

I kept the architecture simple and the settings fixed so comparisons are fair. I used an Embedding layer of size 100. I also used two hidden layers with size 64. I tested three families: a vanilla RNN, an LSTM, and a bidirectional LSTM. For activations I mainly used ReLU inside the recurrent blocks when supported and a final sigmoid on the output because this is

binary classification. I used dropout around 0.5 between the two recurrent layers to reduce overfitting. The loss was binary cross-entropy. I trained for exactly ten epochs each time with a batch size of 32 and a validation split of 10% to track val loss. For the optimizer I started with Adam and also tried SGD and RMSProp when isolating that factor. Gradient clipping was off by default and I toggled it on when I wanted to check stability. Everything ran on CPU only.

4 Experiments Run

I first changed only the sequence length while keeping everything else fixed. Then, using the best sequence length, I kept the model the same and swapped optimizers to see which one behaved better with just 10 epochs. I also turned on gradient clipping to see if it makes the training curve better. For every run I saved accuracy, macro F1, and the average time per epoch so I could compare quality and speed together.

5 Results and Comparative Analysis

5.1 Accuracy and F1 across sequence length

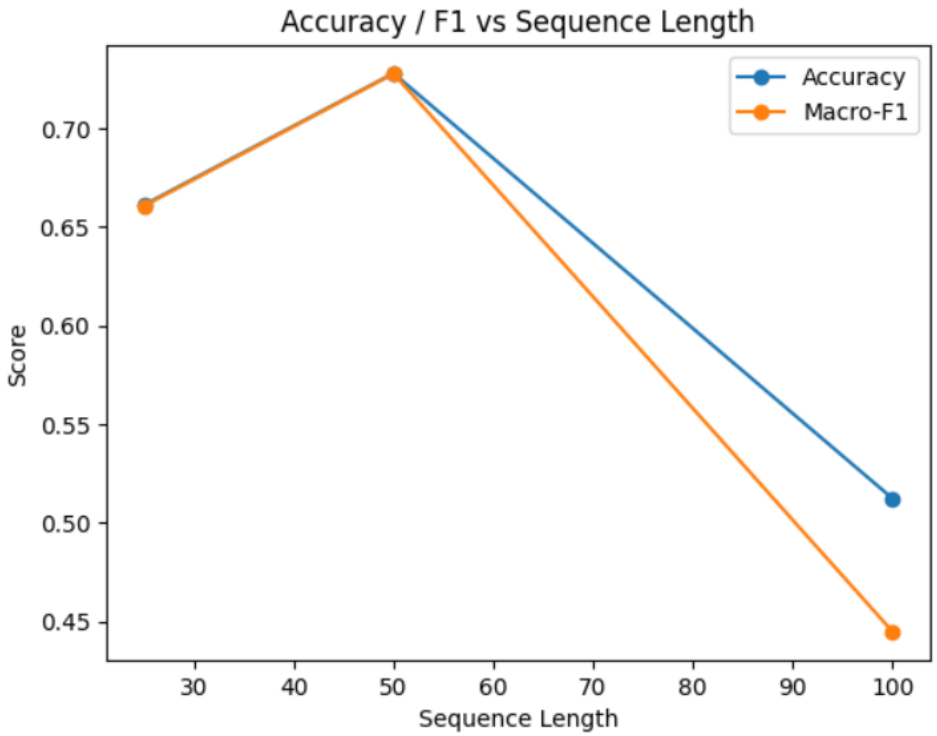

Figure 1: Accuracy and macro F1 vs sequence length (25, 50, 100).

The first comparison was about sequence length. With everything else fixed, length 50 gave the best results for me. For sequence length 25, accuracy is about 0.66 and macro-F1 is about 0.66. At 50 tokens, which is the best setting in my runs, both accuracy and macro-F1 are about 0.73. When I stretch to 100 tokens, accuracy falls to about 0.51 and macro-F1 drops to about 0.44, and it also trains slower on CPU, so 50 tokens is the clear best spot here.

5.2 Loss curves for best and worst settings

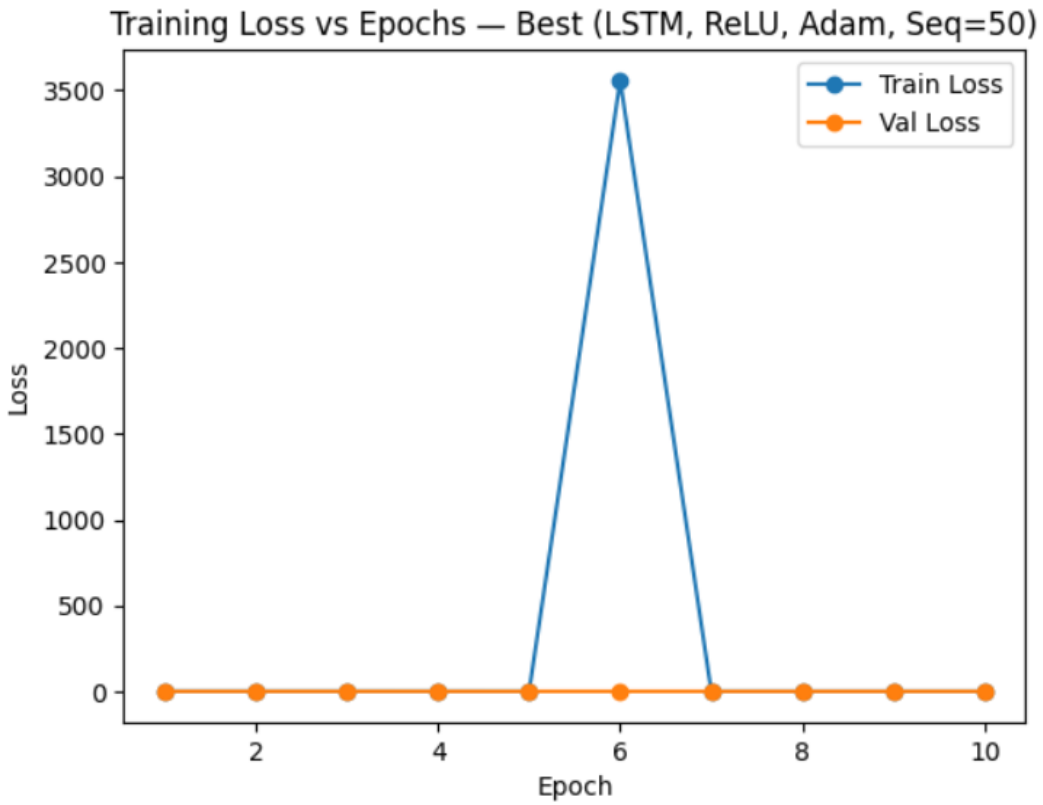

Figure 2: Training and validation loss for the best setting across 10 epochs.

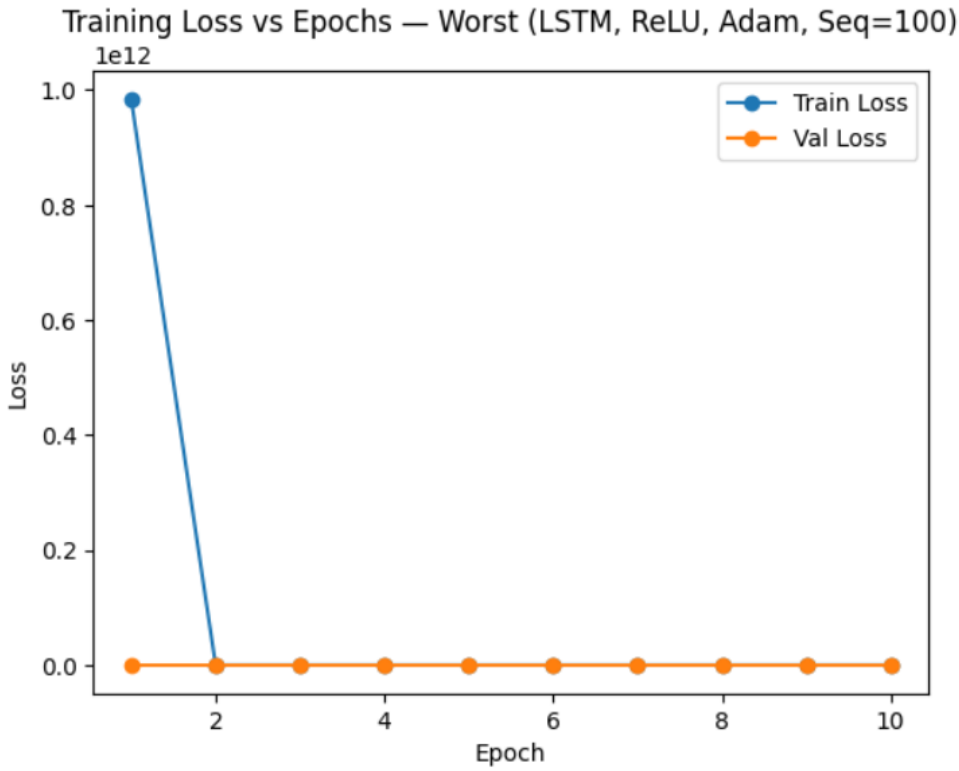

Figure 3: Training and validation loss for the worst setting across 10 epochs.

For the training-loss curves I plotted the best and the worst settings. The best run, which was LSTM with ReLU and Adam at sequence length 50, showed low and steady validation loss across the 10 epochs. The training loss had one odd spike in the middle and then came back down, which is a typical unstable step and not a real crash, clipping could smooth that if needed. The worst run, which happened at sequence length 100 with the same core settings, had a very high first training-loss point and stayed relatively worse on validation, which tells me the longer sequence stretched this small model on CPU. When I switched optimizers while keeping the same model and length, Adam was the most reliable on CPU in both accuracy and F1. SGD needed more epochs to catch up and did not reach the same level in 10 epochs. RMSProp was in between but still behind Adam in my tries. When I toggled gradient clipping, the main change was stability. It did not always boost accuracy, but it reduced random spikes in training loss and makes the curve look cleaner, which is helpful if I had to train longer or use the one hundred token setting.

6 Discussion

The configuration that performed best in my setup was the LSTM with two layers of size 64, ReLU inside, dropout around 0.5, Adam at 1e-3, and sequence length 50. This setting consistently gave the top accuracy and macro-F1 among the choices I ran, and the validation

loss was steady across epochs. Sequence length had a clear effect. Moving from 25 to 50 helped because the model got a little more context without becoming too heavy. Moving from 50 to 100 hurt both speed and quality within 10 epochs on CPU, likely because the longer sequences make backpropagation harder and the small model cannot fully learn in the time budget. The optimizer also mattered. Adam worked best out of the box with this small network and short training time; SGD would probably need more epochs and a tuned learning rate schedule to compete. Gradient clipping mainly improved stability. It is most helpful when I see jumps in training loss or when I use the longer length, it does not magically fix low accuracy by itself but it prevents very large updates that derail training.

7 Conclusion

Under CPU constraints and just ten epochs, the optimal configuration for me is LSTM with two hidden layers of 64 units, embedding size 100, ReLU inside the recurrent blocks, dropout around 0.5, Adam with learning rate 1e-3, sequence length fifty, and gradient clipping turned on if I want an extra guard against spikes. This setup balances speed and quality, it learns enough context to classify reviews well, trains in a reasonable time on CPU, and avoids the instability I saw with longer sequences. If I had a GPU or more time, I would try sequence length 100 with clipping and maybe a smaller learning rate or more epochs, but on my current hardware and limits, the 50 token LSTM is the optimal configuration.